

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Le interfacce

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Le interfacce

<http://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html>

Le interfacce in Java

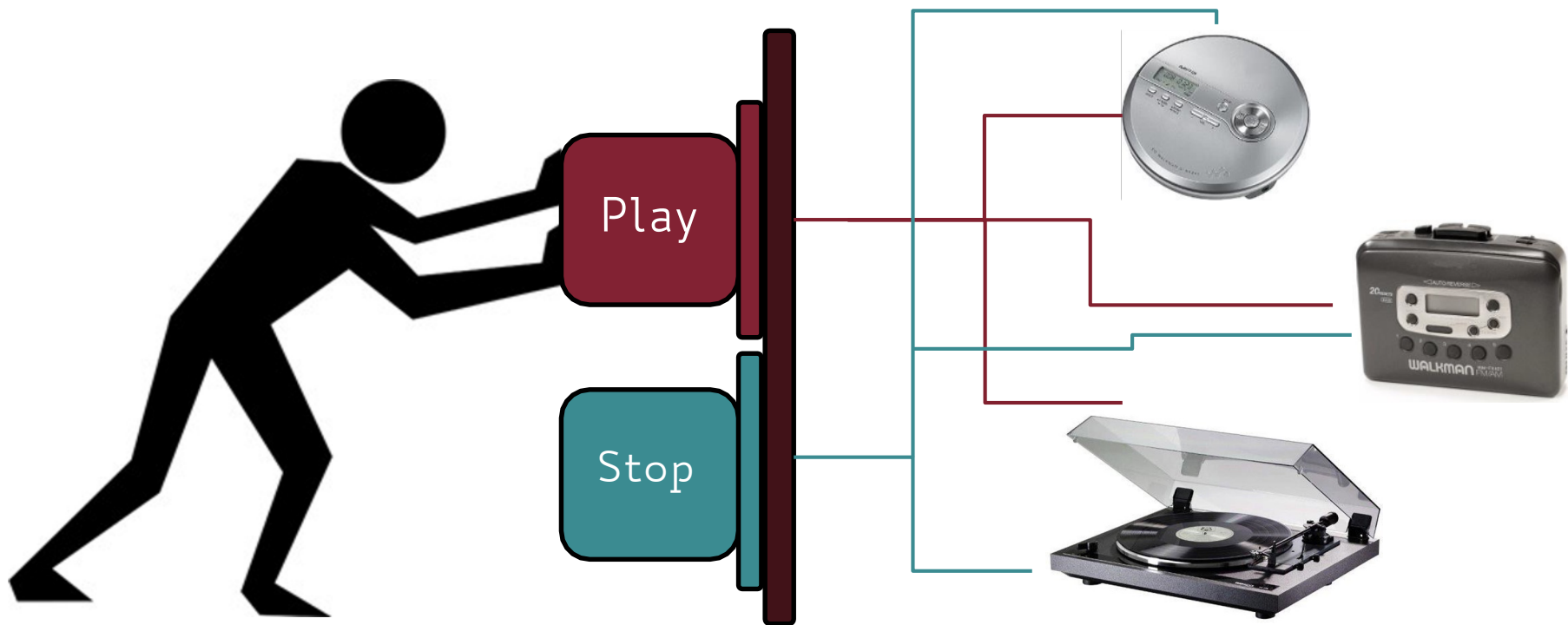
- Le interfacce sono uno strumento che Java mette a disposizione per consentire a più classi di **fornire e implementare un insieme di metodi** comuni
- Le interfacce definiscono e **standardizzano** l'interazione fra oggetti tramite un insieme **limitato** di operazioni

Consideriamo questi dispositivi



Le interfacce in Java

- Esse specificano **soltanto il comportamento** che un certo oggetto deve presentare all'esterno, cioè **cosa** quell'oggetto può fare
- L'**implementazione** di tali operazioni, cioè **come** queste vengono tradotte e realizzate, **rimane** invece **non definito**



Le interfacce in uno slogan

Le interfacce sono classi astratte al 100%



Dichiarazione di un'interfaccia (1)

```
public interface SupportoRiscrivibile
{
    int TIMES = 1000;

    void leggi();
    void scrivi();
}
```

Parola chiave
interface:
indica la
definizione di
una nuova
interfaccia

Costante:
implicitamente
**final, public e
static**

Dichiarazione di
metodi: anche se
non esplicitato,
sono **public** e
abstract

E' **vietato** definire
qualunque
implementazione

Implementare un'interfaccia (2)



```
public class Nastro implements SupportoRiscrivibile
{
    private Pellicola pellicola;

    @Override
    public void leggi()
    {
        attivaTestina();
        muoviTestina();
    }

    @Override
    public void scrivi()
    {
        attivaTestina();
        caricaTestina();
        muoviTestina();
        scaricaTestina();
    }

    public void attivaTestina() {}
    public void caricaTestina() {}
    public void scaricaTestina() {}
    public void muoviTestina() {}
}
```

```
public interface SupportoRiscrivibile
{
    int TIMES = 1000;

    void leggi();
    void scrivi();
}
```

- Per realizzare un'interfaccia è necessario che una classe la implementi, tramite la parola chiave **implements**
- Una classe che implementa una interfaccia decide di voler **esporre pubblicamente** all'esterno il comportamento descritto dall'interfaccia
- E' **obbligatorio** che ciascun metodo abbia esattamente la **stessa intestazione** che esso presenta nell'interfaccia

Implementare un'interfaccia (2)

```
public interface SupportoRiscrivibile
{
    int TIMES = 1000;

    void leggi();
    void scrivi();
}
```

```
public class MemoriaUsb implements SupportoRiscrivibile
{
    private CellaMemoria[] celle;

    @Override
    public void leggi()
    {
        // Leggi la cella corretta
    }

    @Override
    public void scrivi()
    {
        // Modifica la cella corretta
    }
}
```

Anche la classe **MemoriaUsb** implementa l'interfaccia **SupportoRiscrivibile**, definendo i propri metodi **leggi()** e **scrivi()**



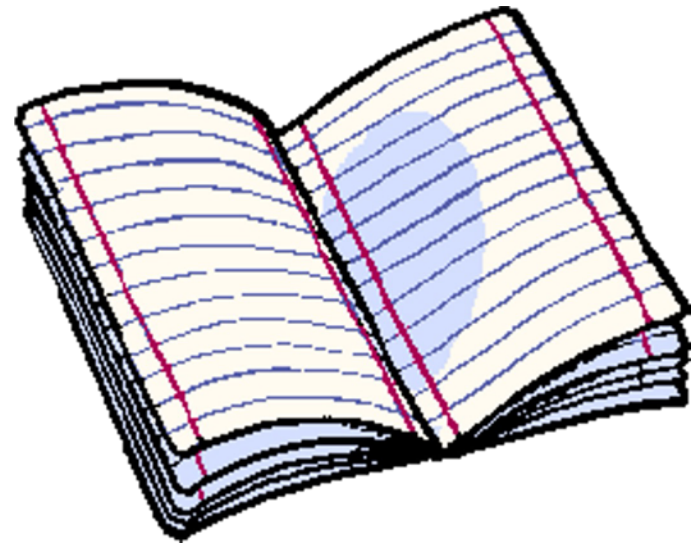
Implementare un'interfaccia (3)

```
public class Quaderno implements SupportoRiscrivibile
{
    private Foglio[] fogli;

    @Override
    public void leggi()
    {
        // Leggi la pagina corrente
    }

    @Override
    public void scrivi()
    {
        // Scrivi sulla pagina corrente
    }
}
```

Perfino un **quaderno** può essere visto come un supporto che è possibile leggere e scrivere!



Le interfacce permettono di **modellare comportamenti comuni** a classi che **non sono necessariamente** in relazione **gerarchica** (is-a, è-un)

Metodi di default e metodi statici (Java 8)

- E' possibile specificare delle implementazioni di default di metodi non statici
 - Mediante la parola chiave `default`
- E implementazioni di metodi statici
 - Sono metodi di utilità non associati alle singole istanze

Metodi privati (Java 9)

- Per facilitare il riuso del codice, da Java 9 è possibile definire metodi privati all'interno di un'interfaccia

```
public interface Loggabile
{
    private void log(String msg, LogLevel level)
    {
        // Codice per il logging
    }

    default void logInfo(String msg) { log(msg, LogLevel.INFO); }
    default void logWarn(String msg) { log(msg, LogLevel.WARNING); }
    default void logError(String msg) { log(msg, LogLevel.ERROR); }
    default void logFatal(String msg) { log(msg, LogLevel.FATAL); }
}
```


Dichiarazione di un'interfaccia (1)

- Un'interfaccia è una **classe** che può contenere soltanto
 - Costanti
 - Metodi astratti
 - Java 8: Implementazione di **default** di metodi e metodi **statici** (utilità)
 - Java 9: metodi **privati** tipicamente da invocare in metodi di default
- Tutti i **metodi** dichiarati in un'interfaccia sono implicitamente **public abstract**
- Tutti i **campi** dichiarati in un'interfaccia sono implicitamente **public static final**
- Tranne nel caso dei metodi di default, statici o privati, **non** è possibile **specificare** alcun dettaglio implementativo
 - non vi è alcun corpo di metodo o variabile di istanza
- Non si possono definire dei Metodi costruttori



Esercizio: Successioni

- Progettare **tre classi** che realizzano diversi tipi di successioni
 - La successione $\{ i^2 \}$ (es. 0, 1, 4, 9, 16, ecc.)
 - La successione **casuale** (es. -42, 2, 5, 18, 154, ecc.)
 - La successione di **Fibonacci** (es. 1, 1, 2, 3, 5, 8, 13, ecc.)
- Elaborare un **meccanismo generale** per la generazione della successione indipendentemente dall'implementazione specifica

Un esempio: l'interfaccia Iterabile (1)

- Ci sono molte classi di **natura diversa** che rappresentano sequenze di elementi
 - Es. **array**, **liste**, **stringhe**, ecc.
- Tuttavia, le sequenze hanno qualcosa in comune: è possibile **iterare sui loro elementi**:

```
public interface Iterabile
{
    boolean hasNext();
    Object next();
    void reset();
}
```

Un esempio: l'interfaccia Iterabile (2)

- Ciascuna classe implementerà i metodi a suo modo:

```
public class MyIntegerArray implements Iterabile
{
    private Integer[] array;
    private int k = 0;

    public MyIntegerArray(Integer[] array)
    {
        this.array = array;
    }

    @Override
    public boolean hasNext()
    {
        return k < array.length;
    }

    @Override
    public Object next()
    {
        return array[k++];
    }

    @Override
    public void reset() { k = 0; }
}
```

Possono essere
specializzati a
**Integer e
Character**

```
public class MyString implements Iterabile
{
    private String s;
    private int k = 0;

    public MyString(String s)
    {
        this.s = s;
    }

    @Override
    public boolean hasNext()
    {
        return k < s.length();
    }

    @Override
    public Object next()
    {
        return s.charAt(k++);
    }

    @Override
    public void reset() { k = 0; }
}
```

**QUESTA
SOLUZIONE
NON VA
BENE!!!!!!**

```
public interface Iterabile
{
    boolean hasNext();
    Object next();
    void reset();
}
```

L'esempio "Iterabile" non è la soluzione ideale per iterare su una collezione

- Perché?
- **Non permette** di mantenere contatori multipli sullo stesso oggetto
- Il seguente esempio **NON FUNZIONA!**

```
MyString s = new MyString("ciao");  
  
while(s.hasNext()){  
    char c1 = s.next();  
    while(s.hasNext()){  
        char c2 = s.next();  
        System.out.println(c1+"-"+c2);  
    }  
}
```

La soluzione: usare le interfacce Iterable e Iterator

- Interfacce standard di Java che disaccoppiano l'oggetto su cui iterare dall'oggetto che tiene la posizione d'iterazione
- `java.lang.Iterable`:

Method Summary

Methods

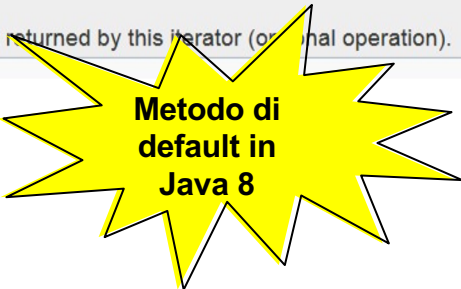
Modifier and Type	Method and Description
<code>Iterator<T></code>	<code>iterator()</code> Returns an iterator over a set of elements of type T.

- `java.util.Iterator`

Method Summary

Methods

Modifier and Type	Method and Description
<code>boolean</code>	<code>hasNext()</code> Returns <code>true</code> if the iteration has more elements.
<code>E</code>	<code>next()</code> Returns the next element in the iteration.
<code>void</code>	<code>remove()</code> Removes from the underlying collection the last element returned by this iterator (optional operation).



Metodo di default in Java 8

L'interfaccia `java.util.Iterator<E>`

- E' un'interfaccia fondamentale che permette di iterare su collezioni
- Espone tre metodi:

Metodo	Descrizione
boolean hasNext()	Restituisce true se esiste ancora un successivo elemento nella collezione
E next()	Restituisce l'elemento successivo
void remove()	Rimuove l'elemento corrente (operazione opzionale)

- E' in relazione con l'interfaccia `Iterable` nel senso che chi implementa `Iterable` restituisce un NUOVO `Iterator` sull'oggetto-collezione

Con Iterable

```
public class IterableIntArray implements Iterable<Integer> {

    private final int[] values;

    public IterableIntArray(int[] values) {
        this.values = values;
    }

    @Override
    public Iterator<Integer> iterator() {

        return new Iterator<Integer>() {

            private int index = 0;

            @Override
            public boolean hasNext() {
                return index < values.length;
            }

            @Override
            public Integer next() {
                int value = values[index];
                index++;
                return value;
            }
        };
    }
}
```




Esercizio: il mio array iterabile

- Scrivere una classe che implementa un array di interi iterabile
- Utilizzare l'interfaccia `Iterable<Integer>`

Esercizio: la mia stringa iterabile

- Scrivere una classe che implementa una stringa iterabile utilizzando `Iterable<Character>`

Esercizio: successioni con iterazione standard

- Implementare l'esercizio sulle successioni utilizzando il meccanismo standard di Java per l'iterazione

Il costrutto "for each"

Se ho

```
IterableIntArray numbers = new IterableIntArray(new int[] { 10, 20, 30, 40 });
```

Per iterare posso utilizzare un notazione compatta:

```
for (int number : numbers) {  
    System.out.println(number);  
}
```

Che equivale a:

```
Iterator<Integer> it=numbers.iterator();  
while (it.hasNext())  
    System.out.println(it.next());
```

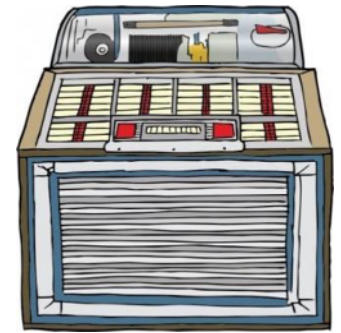
Esempio: Il Jukebox

```
import java.util.ArrayList;
import java.util.Iterator;

/**
 * Rappresenta un JukeBox di canzoni su cui si può iterare
 *
 * @author navigli
 */
public class Jukebox implements Iterable<Canzone>
{
    /**
     * Elenco delle canzoni
     */
    private ArrayList<Canzone> canzoni = new ArrayList<Canzone>();

    /**
     * Permette di aggiungere una canzone
     * @param c la canzone da aggiungere
     */
    public void addCanzone(Canzone c)
    {
        canzoni.add(c);
    }

    @Override
    public Iterator<Canzone> iterator()
    {
        return canzoni.iterator();
    }
}
```



Implementare un'interfaccia: il contratto (1)

- Implementare un'interfaccia equivale a **firmare un contratto** con il compilatore che stabilisce l'impegno ad implementare tutti i metodi specificati dall'interfaccia o a dichiarare la classe **abstract**



Implementare un'interfaccia: il contratto (2)

- 3 possibilità per una classe che implementa un'interfaccia:
 - fornire un'implementazione concreta di **tutti i metodi**, definendone il corpo
 - fornire l'implementazione concreta solo **per un sottoinsieme** proprio dei metodi dell'interfaccia
 - decidere di **non fornire alcuna implementazione** concreta
- Negli ultimi due casi, però, la classe va dichiarata **abstract**

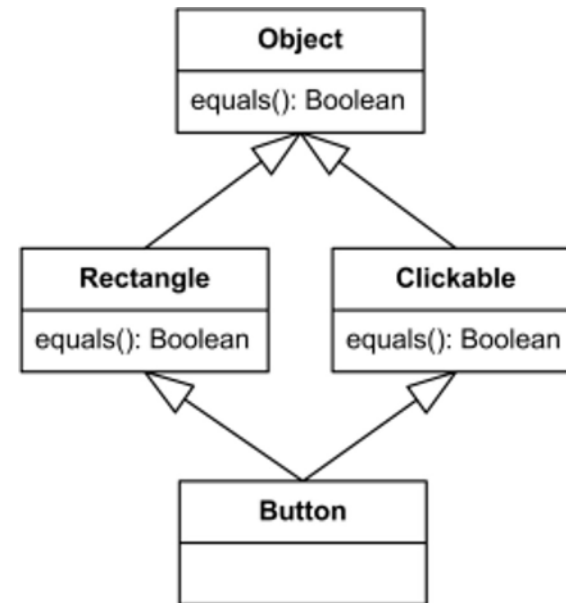
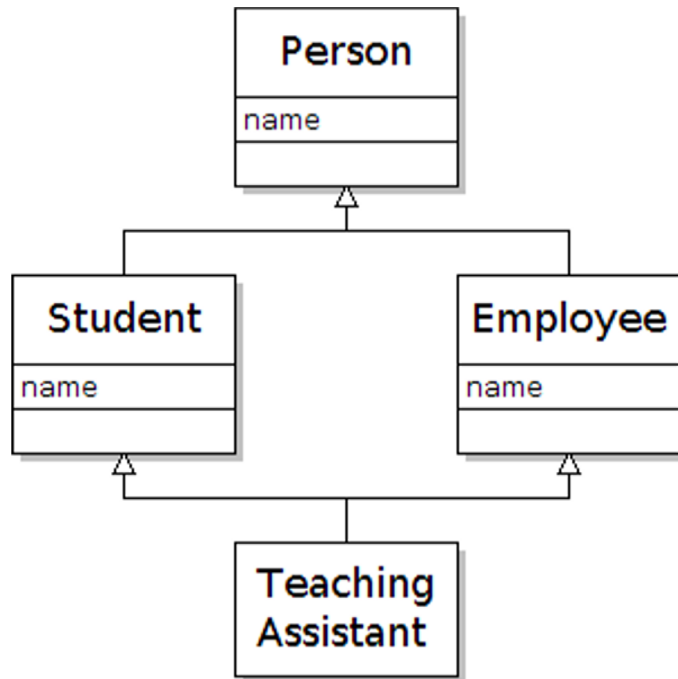
Interfacce vs. classi astratte

- Una domanda nasce spontanea: Se implementando un'interfaccia devo dichiarare tutti i metodi in essa definiti, perché non ricorrere ad una **classe astratta**?



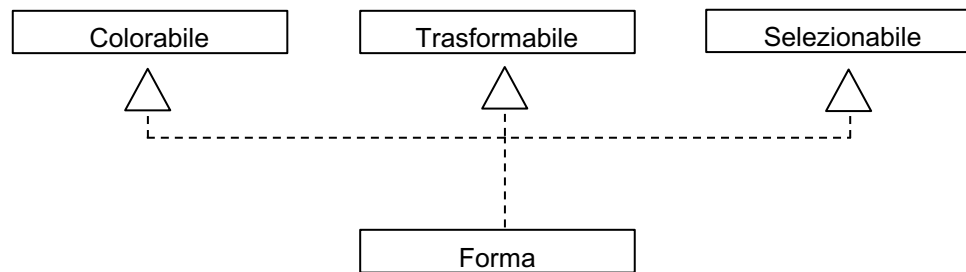
Il problema del diamante

- Con l'**ereditarietà multipla** si possono creare situazioni **poco chiare** di **duplicazione** di metodi e campi



Ereditarietà multipla (1)

- In Java non è consentito estendere più di una classe alla volta: ovvero **extends** può essere seguito solo da un unico nome di classe
- Al contrario, una classe può **implementare** tutte le interfacce desiderate



- E' inoltre possibile estendere 1 classe e contemporaneamente implementare n interfacce diverse

Ereditarietà multipla (2)

```
public class Forma implements Colorabile, Trasformabile, Selezionabile
{
    @Override
    public void seleziona()
    {
    }

    @Override
    public void deseleziona()
    {
    }

    @Override
    public void sposta(Point point)
    {
    }

    @Override
    public void ruota(Angle angle)
    {
    }

    @Override
    public void ritaglia(Point upperLeft, Point bottoRight)
    {
    }

    @Override
    public void scala(double factor)
    {
    }

    @Override
    public void colora(Color c)
    {
    }

    @Override
    public void rimuoviColore()
    {
    }
}
```

La classe **Forma** implementa 3 interfacce e definisce tutti i metodi da esse dichiarati

```
public interface Selezionabile
{
    void seleziona();
    void deseleziona();
}
```

```
public interface Colorabile
{
    void colora(Color c);
    void rimuoviColore();
}
```

```
public interface Trasformabile
{
    void sposta(Point point);
    void ruota(Angle angle);
    void ritaglia(Point upperLeft, Point bottoRight);
    void scala(double factor);
}
```

Relazione tra interfacce e classi che le implementano

- Nel momento in cui una classe **C** decide di implementare un'interfaccia **I**, tra queste due classi si instaura una relazione di tipo **is-a**, ovvero **C è di tipo I**
- Comportamento simile a quello dell'ereditarietà
- Quindi anche per le interfacce **valgono le regole del polimorfismo**
- Ad esempio, la seguente dichiarazione è lecita:

```
SupportoRiscrivibile supporto = new Nastro();  
supporto.leggi();
```

- E ci consente di usare l'oggetto della classe **Nastro** come fosse di tipo **SupportoRiscrivibile**
 - Con conseguente restringimento della visibilità ai soli **metodi dell'interfaccia SupportoRiscrivibile**

Cosa succede se due metodi di default vengono ereditati da due interfacce implementate?

- Se è necessario richiamare quel metodo nella classe che implementa le due interfacce posso "disambiguare" il metodo chiamando quello appropriato con la sintassi

```
void metodoEsempio(){  
  
    Interfaccia.super.metodo(...);  
  
}
```

Interfacce notevoli

Interfaccia	Descrizione
Comparable	Impone un ordinamento naturale degli oggetti tramite il metodo: <code>int compareTo(T b)</code> , che restituisce un valore $>$, $=$ o $<$ 0 se l'oggetto è rispettivamente maggiore, uguale o minore di <code>b</code> (<code>T</code> è un tipo generico; può essere qualsiasi tipo)
Cloneable	Le classi che implementano quest'interfaccia dichiarano al metodo <code>clone()</code> di <code>Object</code> che è legale effettuare una copia campo-a-campo delle istanze della classe Il metodo <code>clone()</code> invocato su oggetti di classi che non implementano <code>Cloneable</code> solleva una <code>CloneNotSupportedException</code>
Serializable	Quest'interfaccia non possiede metodi o campi e serve soltanto ad identificare il fatto che l'oggetto è serializzabile, cioè memorizzabile su un certo supporto

Passare funzioni in input mediante le interfacce

- Le interfacce permettono il passaggio in input di funzioni con una determinata intestazione
- Ad esempio:

```
public interface Runnable
{
    void run();
}
```

Method Summary

All Methods	Instance Methods	Abstract Methods
Modifier and Type	Method and Description	
void	run() When an object implementing interface Runnable is used to create a thread, starting the thread causes the object's run method to be called in that separately executing thread.	

Passare funzioni in input mediante le interfacce

- Posso quindi implementare **Runnable** in quante classi voglio, **definendo diversi comportamenti** per il metodo `void run()`

```
public class Parlare implements Runnable
{
    public void run() { System.out.println("bla bla bla..."); }
}
```

```
public class Dormire implements Runnable
{
    public void run() { System.out.println("zzzzz....."); }
}
```

Le enumerazioni possono estendere interfacce

- A cosa serve? Posso rendere le enumerazioni estensibili quanto voglio!
- Nell'esercizio sulle espressioni matematiche, posso utilizzare costanti enumerative come istanziazioni dell'interfaccia **OperatoreBinario**

```
public interface OperatoreBinario
{
    double applica(double a, double b);
}
```

```
public enum OperatoriDiBase implements OperatoreBinario
{
    SOMMA { public double applica(double a, double b) { return a+b; } },
    SOTTRAZIONE { public double applica(double a, double b) { return a-b; } },
    PRODOTTO { public double applica(double a, double b) { return a*b; } },
    DIVISIONE { public double applica(double a, double b) { return a/b; } }
}
```



Esercizio: Animali

- Progettare una gerarchia di classi dei seguenti animali, ciascuno con determinate caratteristiche:
 - Uccello (vola, becca)
 - Pinguino (becca, nuota)
 - Aquila (vola, becca)
 - Pesce (nuota)
 - Pesce volante (nuota, vola)
 - Cane (salta, corre, fedele a, domestico)
 - Felino (salta, corre, fa le fusa)
 - Gatto (salta, corre, fa le fusa, domestico)
 - Uomo (salta, corre, pensa, nuota, vola?)

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.